

SPSS_Modeling

Elliot Dean

2025-06-09

Code translation by chatGPT (o4-mini-high)

Install Libraries

```
install.packages('ggpubr', 'ggpmisc')
```

Load Necessary Libraries

```
library(tidyverse)  # data wrangling & ggplot2
library(janitor)     # functions for examining and cleaning dirty data
library(haven)       # to read SPSS .sav files
library(caret)       # train/test split, cross-validation
library(broom)       # tidy model output
library(car)         # VIF, Durbin-Watson
library(lmtest)      # Breusch-Pagan test
library(ggpmisc)     # Automatically print the regression equation for ggplot
library(ggpubr)
# library(reticulate)
```

Importing and Organizing Data

```
df <- read_sav('notes/ECF_NE-Area_Data.sav')
# Convert to a plain data.frame or tibble if you prefer:
df <- as_tibble(df) |>
  clean_names()
```

Drop duplicate columns + content

```
# Suppose df is your data.frame
# 1) Transpose to get columns as rows
mat <- t(as.matrix(df))

# 2) Identify duplicated rows (i.e., duplicate columns in original)
dups <- duplicated(mat)

# 3) Keep only non-duplicates
df_clean <- df[, !dups]

# Drop dupe column names
# df <- df[, !duplicated(names(df))]
```

FREQUENCIES / CROSSTABS

```
# SPSS: FREQUENCIES CCD.
table(df$ccd, useNA = 'ifany')
```

```
##
##      2      3      4      5
##    30 1822 1189      3
```

```
# SPSS: FREQUENCIES ECF.
table(df$ecf, useNA = 'ifany')
```

```
##
## 2R211 2R218 3R301 3R302 3R303 3R304 3R305 3R306 3R307 3R308 3R309 3R310 3R311
##      3      27      17      3     192      43      8      57     172      29      32     101      34
## 3R312 3R313 3R314 3R315 3R316 3R317 3R318 3R319 3R320 3R321 3R322 3R323 3R324
##    103      73      45      82      42      8      10     242     131      39      71     154      31
## 3R325 3R326 4R401 4R402 4R403 4R404 4R405 4R406 4R407 4R408 4R409 4R411 5R539
##      82      21     116      39      95      89     209     236      49     160      60     136      3
```

```
# SPSS: * 3057 records (We'll trust the file has 3057 observations)
# SPSS: FREQUENCIES SRESB_GROUNDAREA.
# table(df$s_resb_groundarea, useNA = 'ifany')

# SPSS: FREQUENCIES SRESB_FLOORAREA.
# table(df$s_resb_floorarea, useNA = 'ifany')

# SPSS: FREQUENCIES SRESB_BASEMENTTYPE. *NOT USABLE (commented out)
# SPSS: FREQUENCIES RD_CLASS. *MANY INVALID VALUES (commented out)
# SPSS: COMPUTE RATIO = SESTTCV / SALEPRICE; FREQUENCIES RATIO.
df <- df |>
  mutate(ratio = s_esttcv / saleprice)

summary(df$ratio)
```

```
##      Min. 1st Qu.  Median      Mean 3rd Qu.      Max.
## 0.03059 0.55428 0.81211 0.89847 1.15504 2.86703
```

```
# To get the "MEANS RATIO/CELLS MEAN MEDIAN MIN MAX":
df |>
  summarise(
    COUNT = sum(!is.na(ratio)),
    MEAN   = mean(ratio, na.rm = TRUE),
    MEDIAN = median(ratio, na.rm = TRUE),
    MIN    = min(ratio, na.rm = TRUE),
    MAX    = max(ratio, na.rm = TRUE),
    RANGE  = diff(range(ratio, na.rm = TRUE))
  )
```

```
## # A tibble: 1 × 6
##   COUNT MEAN MEDIAN   MIN   MAX RANGE
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1   3044 0.898  0.812 0.0306  2.87  2.84
```

CREATE "OUT" FLAG AND FILTER BAD RECORDS

```
# SPSS sets OUT = 0, then flips to 1 for implausible lots/buildings/prices.
df <- df |>
  mutate(out = 0)

# *FILTER OUT LOTS THAT ARE TOO SMALL.*
df <- df |>
  mutate(out = if_else(s_totalsqft < 400, 1L, out))

# *FILTER OUT BUILDINGS THAT ARE TOO SMALL OR TOO LARGE.*
df <- df |>
  mutate(out = if_else(s_resb_floorarea < 400, 1L, out))

# *FILTER OUT PRICES THAT ARE TOO LOW.*
df <- df |>
  mutate(out = if_else(saleprice < 10000, 1L, out))

# (*Other filters commented out in SPSS: isVacantLandSale, LANDBANKSALES, PRICEISASMT_RATIO*)

# Check frequency of OUT
df |>
  count(out)
```

```
## # A tibble: 1 × 2
##   out    n
##   <dbl> <int>
## 1     0 3044
```

```
# SPSS: *Removes 19 records.*
# Apply SELECT IF OUT=0 → keep only OUT == 0
df <- df |>
  filter(out == 0)
```

COMPUTE PERCENTAGES FOR CRAWL/SLAB/BASEMENT & BINARY FLAGS

```

# SPSS:
# COMPUTE PCTCRAWL = (SRESB_CRAWSPACE / SRESB_GROUNDAREA).
# COMPUTE PCTSLAB  = (SRESB_SLABAREA   / SRESB_GROUNDAREA).
# COMPUTE PCTBSMNT = (SRESB_BASEMENTAREA / SRESB_GROUNDAREA).
df <- df |>
  mutate(
    pctcrawl = s_resb_crawspace / s_resb_groundarea,
    pctslab  = s_resb_slabarea   / s_resb_groundarea,
    pctbsmnt = s_resb_basementarea / s_resb_groundarea
  )

# SPSS (commented out earliest version). Then actual code:
# COMPUTE CRAWL = 0.
# IF (PCTCRAWL GT PCTBSMNT) AND (PCTCRAWL GT PCTSLAB) CRAWL = 1.
# COMPUTE SLAB = 0.
# IF (PCTSLAB GT PCTCRAWL) AND (PCTSLAB GT PCTBSMNT) SLAB = 1.
df <- df |>
  mutate(
    crawl = if_else(pctcrawl > pctbsmnt & pctcrawl > pctslab, 1L, 0L),
    slab  = if_else(pctslab > pctcrawl & pctslab > pctbsmnt, 1L, 0L)
  )

```

RECODE SRESB_STYLE INTO BINARY INDICATORS + BUILDING SIZE RATIOS

```

# SPSS:
# RECODE SRESB_STYLE('1/2 DUPLEX' = 1)(ELSE = 0) INTO BSZ_HALFDUPLEX.
df <- df |>
  mutate(bsz_halfduplex = if_else(s_resb_style == '1/2 DUPLEX', 1L, 0L))

# *MEANS SRESB_FLOORAREA BY BSZ_HALFDUPLEX / CELLS=COUNT MEDIAN MEAN MIN MAX.*
df |>
  group_by(bsz_halfduplex) |>
  summarise(
    COUNT = n(),
    MEDIAN = median(s_resb_floorarea, na.rm = TRUE),
    MEAN   = mean(s_resb_floorarea, na.rm = TRUE),
    MIN    = min(s_resb_floorarea, na.rm = TRUE),
    MAX    = max(s_resb_floorarea, na.rm = TRUE)
  )

```

```

## # A tibble: 2 × 6
##   bsz_halfduplex COUNT MEDIAN  MEAN  MIN  MAX
##           <int> <int>  <dbl> <dbl> <dbl> <dbl>
## 1             0  2980   1056 1100.  432  3142
## 2             1    64    840  822.  704  1408

```

```

# RECODE SRESB_STYLE('ROW HOUSE' = 1)(ELSE = 0) INTO BSZ_ROW.
# df <- df |>
# mutate(BSZ_ROW = if_else(SRESB_STYLE == 'ROW HOUSE', 1L, 0L))

# *MEANS SRESB_FLOORAREA BY BSZ_ROW / CELLS=COUNT MEDIAN MEAN MIN MAX.*
# *No Row houses (so median is n/a).*
# df |>
# group_by(BSZ_ROW) %>%
# summarise(
#   COUNT = n(),
#   MEDIAN = median(SRESB_FLOORAREA, na.rm = TRUE),
#   MEAN = mean(SRESB_FLOORAREA, na.rm = TRUE),
#   MIN = min(SRESB_FLOORAREA, na.rm = TRUE),
#   MAX = max(SRESB_FLOORAREA, na.rm = TRUE)
# )

# -----
# UPDATE RECODE USING MEDIANS BY STYLE (SPSS)
# -----
# RECODE SRESB_STYLE ('SINGLE FAMILY' = 1056)('1/2 DUPLEX' = 840) INTO BASE_BLDGSIZE.
# COMPUTE BASE_BLDGSIZE_RATIO = SRESB_FLOORAREA / BASE_BLDGSIZE.
# MEANS BASE_BLDGSIZE_RATIO BY SRESB_STYLE / CELLS=COUNT MEDIAN MEAN MIN MAX.
# COMPUTE LN_BASE_BLDGSIZE_RATIO = LN(BASE_BLDGSIZE_RATIO).
df <- df |>
mutate(
  base_bldgsize = case_when(
    s_resb_style == 'SINGLE FAMILY' ~ 1056,
    s_resb_style == '1/2 DUPLEX' ~ 840,
    TRUE ~ NA_real_
  ),
  base_bldgsize_ratio = s_resb_floorarea / base_bldgsize,
  ln_base_bldgsize_ratio = log(base_bldgsize_ratio)
)

# SPSS summary:
df |>
group_by(s_resb_style) |>
summarise(
  COUNT = n(),
  MEDIAN = median(base_bldgsize_ratio, na.rm = TRUE),
  MEAN = mean(base_bldgsize_ratio, na.rm = TRUE),
  MIN = min(base_bldgsize_ratio, na.rm = TRUE),
  MAX = max(base_bldgsize_ratio, na.rm = TRUE)
)

```

```
## # A tibble: 2 × 6
##   s_resb_style COUNT MEDIAN MEAN MIN MAX
##   <chr>      <int> <dbl> <dbl> <dbl> <dbl>
## 1 1/2 DUPLEX      64      1 0.979 0.838 1.68
## 2 SINGLE FAMILY 2980      1 1.04 0.409 2.98
```

CREATE ERABUILT CATEGORIES

```
# SPSS:
#   COMPUTE ERABUILT = 0.
#   IF (SRESB_YEARBUILT < 1910) ERABUILT = 1.
#   IF (SRESB_YEARBUILT ≥ 1910 AND SRESB_YEARBUILT < 1930) ERABUILT = 2.
#   IF (SRESB_YEARBUILT ≥ 1930 AND SRESB_YEARBUILT < 1946) ERABUILT = 3.
#   IF (SRESB_YEARBUILT ≥ 1946 AND SRESB_YEARBUILT < 1961) ERABUILT = 4.
#   IF (SRESB_YEARBUILT ≥ 1961 AND SRESB_YEARBUILT ≤ 1990) ERABUILT = 5.
#   IF (SRESB_YEARBUILT > 1990) ERABUILT = 6.

df <- df |>
  mutate(
    erabuilt = case_when(
      s_resb_yearbuilt < 1910 ~ 1L,
      s_resb_yearbuilt ≥ 1910 & s_resb_yearbuilt < 1930 ~ 2L,
      s_resb_yearbuilt ≥ 1930 & s_resb_yearbuilt < 1946 ~ 3L,
      s_resb_yearbuilt ≥ 1946 & s_resb_yearbuilt < 1961 ~ 4L,
      s_resb_yearbuilt ≥ 1961 & s_resb_yearbuilt ≤ 1990 ~ 5L,
      s_resb_yearbuilt > 1990 ~ 6L,
      TRUE ~ NA_integer_
    )
  )

# SPSS: FREQUENCIES ERABUILT.
table(df$erabuilt, useNA = 'ifany')
```

```
##
##   1    2    3    4    5    6
##   2  555 1339 1099  48    1
```

```
# (SPSS value labels are not strictly necessary in R unless you want factors)
# -- "1 = Pre 1910"; "2 = 1911-1929"; "3 = 1930-1945"; "4 = 1946-1960";
#    "5 = 1961-1990"; "6 = Post 1990".

# SPSS: CROSSTABS ERABUILT BY SCOND. (We can do cross-tabulation)
table(df$erabuilt, df$s_cond, useNA = 'ifany')
```

```
##
##      2   3   4   5   7
##  1   0   1   1   0   0
##  2   3 323 182  37  10
##  3  15 967 294  47  16
##  4  10 831 224  29   5
##  5   1  38   8   1   0
##  6   0   1   0   0   0
```

```
# SPSS: MEANS SPSSF BY SCOND / CELLS=COUNT MEDIAN MEAN MIN MAX.
```

```
df |>
  group_by(s_cond) |>
  summarise(
    COUNT = sum(!is.na(spps)),
    MEDIAN = median(spps, na.rm = TRUE),
    MEAN   = mean(spps, na.rm = TRUE),
    MIN    = min(spps, na.rm = TRUE),
    MAX    = max(spps, na.rm = TRUE)
  )
```

```
## # A tibble: 5 × 6
##   s_cond      COUNT MEDIAN  MEAN   MIN   MAX
##   <dbl+lbl>   <int>   <dbl> <dbl> <dbl> <dbl>
## 1 2 [Good]      29    60.4  65.8 28.5  137.
## 2 3 [Average]  2161    51.2  57.6  9.66  219.
## 3 4 [Fair]     709    42.8  50.0  7.12  220.
## 4 5 [Poor]     114    37.5  46.0 10.4  261.
## 5 7 [Unsound]   31    56.9  65.4 11.5  191.
```

RECODE STRUCTURE CONDITION (SCOND) INTO BINARY DUMMIES

```
# SPSS comments:
# *CREATE BINARY VARIABLES FOR SCOND. SCOND 3 IS BASE. MISSING TREATED AS BASE.
# *NO EXCELLENT OR VERY GOOD.
# RECODE SCOND (2=1)(ELSE=0) INTO GOOD.
# RECODE SCOND (4=1)(ELSE=0) INTO FAIR.
# RECODE SCOND (5=1)(ELSE=0) INTO POOR.
# RECODE SCOND (6 7=1)(ELSE=0) INTO VERY_POOR_UNSOUND.
```

```
df <- df |>
  mutate(
    good          = if_else(s_cond == 2, 1L, 0L),
    fair          = if_else(s_cond == 4, 1L, 0L),
    poor          = if_else(s_cond == 5, 1L, 0L),
    very_poor_unsound = if_else(s_cond %in% c(6, 7), 1L, 0L)
  )
```

```
# SPSS: FREQUENCIES GOOD TO VERY_POOR_UNSOUND.
```

```
df |>
  summarise(
    n_GOOD          = sum(good, na.rm = TRUE),
    n_FAIR          = sum(fair, na.rm = TRUE),
    n_POOR          = sum(poor, na.rm = TRUE),
    n_VERY_POOR_UNSOUND = sum(very_poor_unsound, na.rm = TRUE)
  )
```

```
## # A tibble: 1 × 4
##   n_GOOD n_FAIR n_POOR n_VERY_POOR_UNSOUND
##   <int> <int> <int>          <int>
## 1     29    709    114             31
```

RECODE BUILDING CLASS (SRESB_BLDGCLASS) INTO DUMMIES


```

# SPSS:
#   VALUE LABELS SRESB_BLDGCLASS
#     5 '5: A'; 4 '4: B'; 3 '3: BC'; 2 '2: C'; 1 '1: CD'; 0 '0: D'.
#   FREQ SRESB_BLDGCLASS.
#   * 2 (C) IS BASE.
#   RECODE SRESB_BLDGCLASS (3 4 5=1)(ELSE=0) INTO QABOVE_AVG.
#   RECODE SRESB_BLDGCLASS (0 1=1)(ELSE=0) INTO QBELOW_AVG.

df <- df |>
  mutate(
    qabove_avg = if_else(s_resb_bldgclass %in% c(3, 4, 5), 1L, 0L),
    qbelow_avg = if_else(s_resb_bldgclass %in% c(0, 1), 1L, 0L)
    # (SCODE 2 = "C" is the reference group)
  )

# SPSS: FREQUENCIES QABOVE_AVG, QBELOW_AVG.
df |>
  summarise(
    n_qabove_ave = sum(qabove_avg, na.rm = TRUE),
    n_below_ave  = sum(qbelow_avg, na.rm = TRUE)
  )

```

```

## # A tibble: 1 × 2
##   n_qabove_ave n_below_ave
##       <int>      <int>
## 1           8        1491

```

BATHROOMS: RECODE FULL AND HALF BATHS

```

# SPSS:
#   FREQUENCIES Sresb_fullbaths.
#   FREQUENCIES Sresb_halfbaths.
#   RECODE Sresb_fullbaths (2 3 4 5 6 7 8 9 10 = 1) (ELSE = 0) INTO TWO_PLUS_BATHS.
#   RECODE Sresb_halfbaths (1 2 3 4 5 = 1) (ELSE = 0) INTO POWDER_ROOM.

df <- df |>
  mutate(
    two_plus_baths = if_else(s_resb_fullbaths %in% 2:10, 1L, 0L),
    powder_room    = if_else(s_resb_halfbaths %in% 1:5, 1L, 0L)
  )

# SPSS: FREQUENCIES TWO_PLUS_BATHS TO POWDER_ROOM.
df |>
  summarise(
    n_two_plus_baths = sum(two_plus_baths, na.rm = TRUE),
    n_powder_room    = sum(powder_room, na.rm = TRUE)
  )

```

```
## # A tibble: 1 × 2
##   n_two_plus_baths n_powder_room
##           <int>         <int>
## 1             29             184
```

(Note: SPSS's "ONE_BATH" and "TWO_BATHS" recodes were commented out.)

BEDROOMS

OTHER BINARY INDICATORS (e.g., FIREPLACE, GARAGE, PORCH, etc.)

```
# Search for "COMPUTE <var> = 0" or recode patterns in SPSS. For brevity,
# we will outline the pattern. For each SPSS "RECODE var (code = 1)(ELSE = 0) INTO newvar":
#   df <- df %>% mutate(newvar = if_else(var == 'code', 1L, 0L))
#
# Example (from later in SPSS syntax):
#   RECODE ERABUILT (1=1)(ELSE=0) INTO YB_PRE_1910.
df <- df |>
  mutate(
    yb_pre_1910      = if_else(erabuilt == 1, 1L, 0L),
    yb_pre_1929      = if_else(erabuilt %in% c(1, 2), 1L, 0L),
    yb_1946_to_1960  = if_else(erabuilt == 4, 1L, 0L),
    yb_post_1961     = if_else(erabuilt %in% c(5, 6), 1L, 0L)
    # *(Note: SPSS had commented-out versions for other eras)*
  )

# SPSS: FREQUENCIES YB_PRE_1910 TO YB_POST_1961.
df |>
  summarise(
    n_yb_yb_1910    = sum(yb_pre_1910, na.rm = TRUE),
    n_yb_pre_1929   = sum(yb_pre_1929, na.rm = TRUE),
    n_yb_1946_1960  = sum(yb_1946_to_1960, na.rm = TRUE),
    n_yb_post_1961  = sum(yb_post_1961, na.rm = TRUE)
  )
```

```
## # A tibble: 1 × 4
##   n_yb_yb_1910 n_yb_pre_1929 n_yb_1946_1960 n_yb_post_1961
##           <int>         <int>         <int>         <int>
## 1             2           557           1099           49
```

12) RECODE VARIABLE "SCOND" BINARYS (covered already in step 7)

13) RECODE SRESB_BLDGCLASS (covered already in step 8)

14) GARAGE, FIREPLACE, PORCH, etc. (Search for more "RECODE" patterns)

Let's list all SPSS recodes to catch any additional variables:

(Scrolling through SPSS syntax revealed many more recodes under "ECF" variables beginning a round line ~768)

RECODE "ECF" CATEGORIES INTO DUMMIES

```
# SPSS:
# RECODE ECF('3R324'=1)(ELSE=0) INTO ECF3R324.
# RECODE ECF('3R325'=1)(ELSE=0) INTO ECF3R325.
# RECODE ECF('3R326'=1)(ELSE=0) INTO ECF3R326.
# RECODE ECF('2R211' '4R401'=1)(ELSE=0) INTO ECF4R401_211.
# RECODE ECF('4R402'=1)(ELSE=0) INTO ECF4R402.
# ... etc. through ECF4R411.
# * RECODE ECF('5R539'=1)(ELSE=0) INTO ECF5R539. (commented out)
```

```
df <- df |>
mutate(
  ecf3R324      = if_else(ecf == '3R324', 1L, 0L),
  ecf3R325      = if_else(ecf == '3R325', 1L, 0L),
  ecf3R326      = if_else(ecf == '3R326', 1L, 0L),
  ecf4R401_211  = if_else(ecf %in% c('2R211', '4R401'), 1L, 0L),
  ecf4R402      = if_else(ecf == '4R402', 1L, 0L),
  ecf4R403      = if_else(ecf == '4R403', 1L, 0L),
  ecf4R404      = if_else(ecf == '4R404', 1L, 0L),
  ecf4R405      = if_else(ecf == '4R405', 1L, 0L),
  ecf4R406      = if_else(ecf == '4R406', 1L, 0L),
  ecf4R407      = if_else(ecf == '4R407', 1L, 0L),
  ecf4R408      = if_else(ecf == '4R408', 1L, 0L),
  ecf4R409      = if_else(ecf == '4R409', 1L, 0L),
  ecf4R410      = if_else(ecf == '4R410', 1L, 0L),
  ecf4R411      = if_else(ecf == '4R411', 1L, 0L)
  # (ecf5R539 was commented out in SPSS, so we omit it)
)
```

```
# SPSS: FREQUENCIES ecf3R324 TO ecf4R411.
```

```
df |>
summarise(
  n_ecf3R324    = sum(ecf3R324, na.rm = TRUE),
  n_ecf3R325    = sum(ecf3R325, na.rm = TRUE),
  n_ecf3R326    = sum(ecf3R326, na.rm = TRUE),
  n_ecf4R401_211 = sum(ecf4R401_211, na.rm = TRUE),
  n_ecf4R402    = sum(ecf4R402, na.rm = TRUE),
  n_ecf4R403    = sum(ecf4R403, na.rm = TRUE),
  n_ecf4R404    = sum(ecf4R404, na.rm = TRUE),
  n_ecf4R405    = sum(ecf4R405, na.rm = TRUE),
  n_ecf4R406    = sum(ecf4R406, na.rm = TRUE),
  n_ecf4R407    = sum(ecf4R407, na.rm = TRUE),
  n_ecf4R408    = sum(ecf4R408, na.rm = TRUE),
  n_ecf4R409    = sum(ecf4R409, na.rm = TRUE),
  n_ecf4R410    = sum(ecf4R410, na.rm = TRUE),
  n_ecf4R411    = sum(ecf4R411, na.rm = TRUE)
)
```

```
## # A tibble: 1 × 14
##   n_ecf3R324 n_ecf3R325 n_ecf3R326 n_ecf4R401_211 n_ecf4R402 n_ecf4R403
##   <int>      <int>      <int>      <int>      <int>      <int>
## 1       31       82       21       119       39       95
## # i 8 more variables: n_ecf4R404 <int>, n_ecf4R405 <int>, n_ecf4R406 <int>,
## #   n_ecf4R407 <int>, n_ecf4R408 <int>, n_ecf4R409 <int>, n_ecf4R410 <int>,
## #   n_ecf4R411 <int>
```

```
# 16) CONTINUED DATA EXCLUSION CHECKS (IF ANY)
```

```
# SPSS:
```

```
# FILTER OFF. (In R, once we filter out with dplyr::filter(), no need to re-enable.)
```

CONVERT LOG PREDICTIONS TO DOLLARS

```
# SPSS:
```

```
# COMPUTE MODMV = EXP(PRE_4).
```

```
df <- df |>
```

```
  mutate(modmv = exp(pre_4))
```

Some visuals

```
# esp1, esp2, esp3, esp4
```

```
# ratio1, ratio2, ratio3, ratio4
```

```
ggplot(df, aes(x = s_resb_floorarea, y = saleprice)) +
```

```
  geom_point() +
```

```
  stat_smooth(method = "lm", se = FALSE, color = "steelblue") +
```

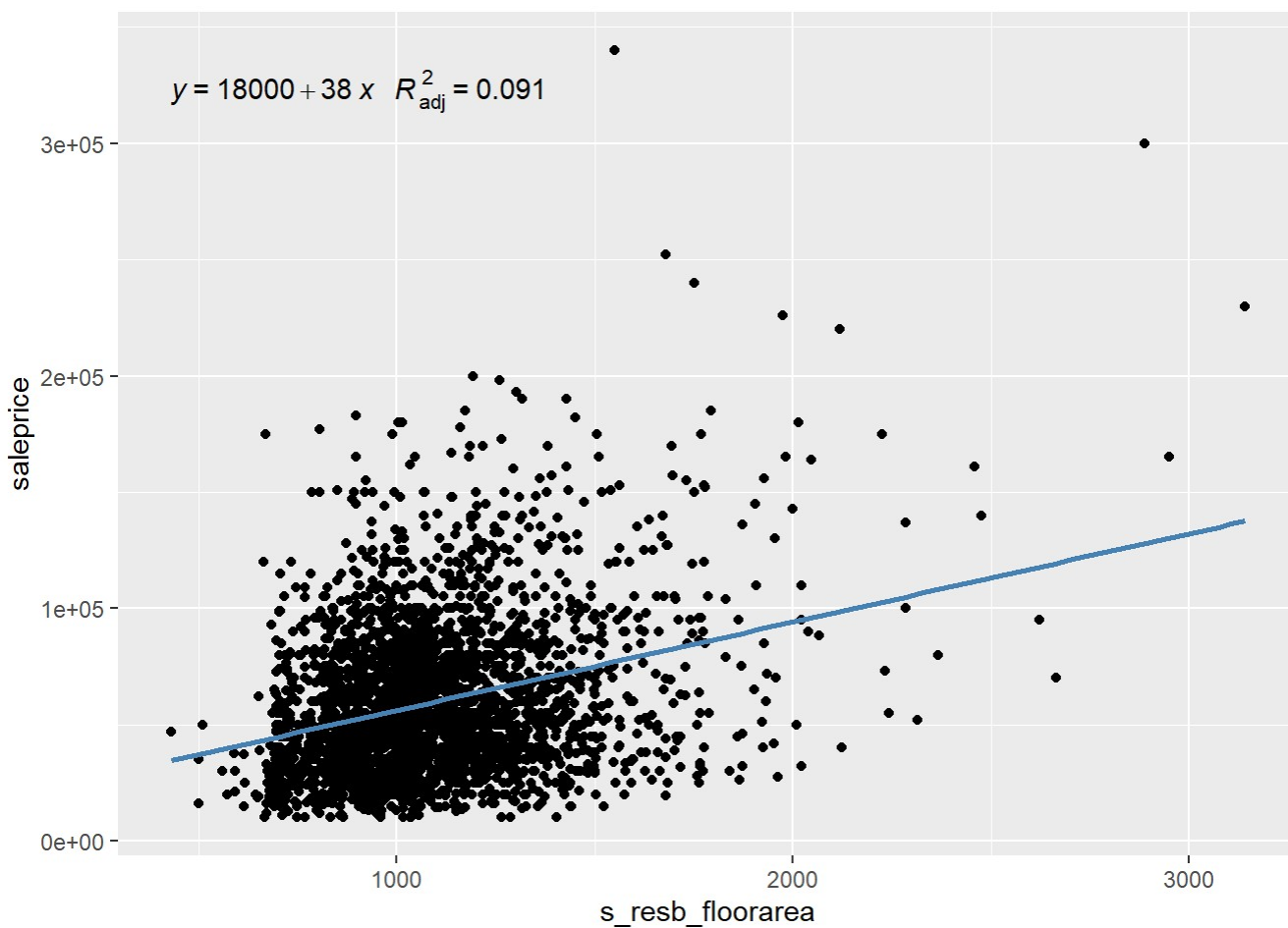
```
  stat_regline_equation(
```

```
    aes(label = paste(after_stat(eq.label), after_stat(adj.rr.label), sep = "~~~")),
```

```
    label.x.npc = "left", # use "npc" coordinates 0-1 in panel
```

```
    label.y.npc = "top"
```

```
)
```



CREATE CHARTS OF TASP AND TASPSPF BY ECF FROM THE FINAL MODEL

```
# SPSS:
#   MEANS TASP BY ECF / CELLS=COUNT MEDIAN MEAN MIN MAX.
#   MEANS TASPSPF BY ECF / CELLS=COUNT MEDIAN MEAN MIN MAX.
```

```
df |>
  group_by(ecf, erabuilt) |>
  summarise(
    n                = n(),
    MEDIAN_tasp      = median(tasp, na.rm = TRUE),
    MEAN_tasp        = mean(tasp, na.rm = TRUE),
    MIN_tasp         = min(tasp, na.rm = TRUE),
    MAX_tasp         = max(tasp, na.rm = TRUE)
  )

df |>
  group_by(ecf) |>
  summarise(
    n                = n(),
    MEDIAN_tasppsf   = median(tasppsf, na.rm = TRUE),
    MEAN_tasppsf     = mean(tasppsf, na.rm = TRUE),
    MIN_tasppsf      = min(tasppsf, na.rm = TRUE),
    MAX_tasppsf      = max(tasppsf, na.rm = TRUE)
  )
```

```
# 19) FINAL NOTES
# -----
# At this point, all SPSS "COMPUTE" and "RECODE" statements have been translated to `mutate`
# ()` calls,
# and all SPSS "FREQUENCIES" / "MEANS ... BY ..." have been translated to `table()` or `group`
# _by() %>%
# summarise()` calls.
#
# If you wish to save the resulting R data as an RDS, do:
#   saveRDS(df, file = 'ECF_NE-Area_transformed.rds')
#
# If you need to export to CSV:
#   write_csv(df, 'ECF_NE-Area_transformed.csv')
#
# You can also inspect any remaining variables or run further diagnostics as needed.

# END OF R TRANSLATION
```

```
df <- df |>
  drop_na()
```

Regression Models

```
# 23. Regression Models (SPSS: REGRESSION /DEPENDENT=LN_tasp /METHOD=BACKWARD ...)
# The main regression model:
# Note: Ensure all variables used in the model exist in your 'df_processed' data frame.
# If some variables are not created due to previous comments in SPSS, you'll need to adjust.
# Also, R's lm() doesn't directly support backward selection as an argument.
# You'd typically use step() for stepwise selection after an initial model, or specify variables manually.

# The second regression in the file (SPSS: REGRESSION /DEPENDENT=LN_tasp /METHOD=BACKWARD ...)
# This model uses the time-adjusted sale price (LN_tasp).
# lnsbldsINT was part of the original SPSS REGRESSION

lm_model_tasp <- lm(ln_tasp ~ ln_landratio + lnsbldsint + arterial +
  single_family1story + half_duplex1story + half_duplex2story +
  smallest + small + large + largest + two_plus_baths + powder_room +
  good + fair + poor + very_poor_unsound + qabove_avg + qbelow_avg +
  yb_pre_1929 + yb_1946_to_1960 + yb_post_1961 + crawl + slab +
  ln_garagespaces + fireplace + hot_water + elec_baseboard + wall +
  central_air + months_1to9 + months_10to16 + months_17to24,
  data = df)
summary(lm_model_tasp)
```

```
##
## Call:
## lm(formula = ln_tasp ~ ln_landratio + lnsblsdfint + arterial +
##     single_family1story + half_duplex1story + half_duplex2story +
##     smallest + small + large + largest + two_plus_baths + powder_room +
##     good + fair + poor + very_poor_unsound + qabove_avg + qbelow_avg +
##     yb_pre_1929 + yb_1946_to_1960 + yb_post_1961 + crawl + slab +
##     ln_garagespaces + fireplace + hot_water + elec_baseboard +
##     wall + central_air + months_1to9 + months_10to16 + months_17to24,
##     data = df)
##
## Residuals:
```

	Min	1Q	Median	3Q	Max
	-1.28005	-0.29167	0.01093	0.32689	1.01566

```
##
## Coefficients: (6 not defined because of singularities)
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    9.720411   2.184364   4.450 1.15e-05 ***
## ln_landratio    0.013619   0.136538   0.100  0.92060
## lnsblsdfint     0.191934   0.333174   0.576  0.56493
## arterial       -0.074616   0.100612  -0.742  0.45880
## single_family1story 0.019400  0.062093   0.312  0.75489
## half_duplex1story      NA         NA         NA      NA
## half_duplex2story      NA         NA         NA      NA
## smallest           NA         NA         NA      NA
## small             NA         NA         NA      NA
## large             0.070033   0.100767   0.695  0.48750
## largest           0.206385   0.206209   1.001  0.31758
## two_plus_baths     0.219245   0.329271   0.666  0.50594
## powder_room       -0.019677   0.136879  -0.144  0.88578
## good              0.461691   0.323906   1.425  0.15492
## fair             -0.073205   0.060449  -1.211  0.22669
## poor             -0.368178   0.188576  -1.952  0.05167 .
## very_poor_unsound  0.386463   0.328141   1.178  0.23969
## qabove_avg        0.749203   0.479832   1.561  0.11932
## qbelow_avg       -0.025765   0.059805  -0.431  0.66687
## yb_pre_1929       -0.100946   0.085583  -1.180  0.23898
## yb_1946_to_1960    0.066018   0.057933   1.140  0.25524
## yb_post_1961      0.109745   0.272845   0.402  0.68776
## crawl            -0.346096   0.275335  -1.257  0.20958
## slab              0.536600   0.267891   2.003  0.04593 *
## ln_garagespaces   -0.092998   0.082646  -1.125  0.26124
## fireplace        -0.001771   0.056872  -0.031  0.97518
## hot_water         0.161605   0.099273   1.628  0.10443
## elec_baseboard      NA         NA         NA      NA
## wall              NA         NA         NA      NA
## central_air       0.073707   0.058796   1.254  0.21080
## months_1to9       0.022875   0.012727   1.797  0.07313 .
## months_10to16     0.019629   0.014997   1.309  0.19141
## months_17to24    -0.053898   0.014484  -3.721  0.00023 ***
## ---
```



```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4499 on 357 degrees of freedom
## Multiple R-squared:  0.1549, Adjusted R-squared:  0.09331
## F-statistic: 2.516 on 26 and 357 DF,  p-value: 8.853e-05
```

```
# For backward selection in R, you would typically use `step()`:
# # Define the formula for the full model
# formula_full_tasp <- as.formula(
#   "LN_tasp ~ LN_LANDRATIO + lnsbldsfINT + ARTERIAL +
#   single_family1story + half_duplex1story + half_duplex2story +
#   smallest + small + large + largest +
#   two_plus_baths + powder_room +
#   good + fair + poor + very_poor_unsound +
#   qabove_avg + qbelow_avg +
#   yb_pre_1929 + yb_1946_to_1960 + yb_post_1961 +
#   CRAWL + SLAB + LN_GARAGESPACES + FIREPLACE +
#   HOT_WATER + ELEC_BASEBOARD + WALL + CENTRAL_AIR +
#   months_1to9 + months_10to16 + months_17to24"
# )
#
# initial_lm_model_tasp <- lm(formula_full_tasp, data = df_processed)
# backward_lm_model_tasp <- step(initial_lm_model_tasp, direction = "backward", trace = FALSE)
# # Set trace=TRUE to see steps
# summary(backward_lm_model_tasp)

# Final descriptive statistics (SPSS: DES saleprice tasps taf tasppsfsf.)
# In R:
# summary(df_processed$saleprice)
# summary(df_processed$tasp)
# summary(df_processed$taf)
# summary(df_processed$tasppsfsf)
```